Beyond Accuracy@k: How Localization Quality Shapes Repository-Level LLM Program Repair

Juan Javier Samuel Z (东万) · L25020002 · October 2026

1. Context

SWE-bench [3] has become the standard test for LLM-based program repair. The model gets a real GitHub issue and a whole codebase, and it has to produce a fix. Studies of this setting keep pointing to the same problem, and it appears before any code is written: the model first has to work out where to edit. MAGIS [2] found that "locating lines is a key factor for GitHub issue resolution," and that issues touching more files and functions are harder to resolve. The latest LLM4APR review [1] states it plainly: "fault localization is a precondition of patch generation" (§9.3.2). This step is also expensive. According to SHERLOC [7], repair agents spend roughly half their budget finding the fault before they make a single edit. This statement focuses on that step.

2. What Has Been Done

As a result, issue localization is now studied as a task of its own. Agentless [10] narrows the search step by step, from files to functions to lines. LocAgent [4] builds a code graph, CoSIL [5] searches the call graph iteratively, and SweRank [6] retrieves candidates and then reranks them. SHERLOC [7] returns a structured diagnosis instead of just a location.

Most of these tools are scored with retrieval metrics such as Acc@k and Recall@k. When papers report how much they help repair, they usually give the end-to-end gain. For example, adding CoSIL to Agentless raises the resolution rate by 2.98–30.5% [5]. Loc2Repair [8] was the first to measure the two steps separately under the same conditions. On SWE-bench Verified, predicted file-level localization raised the pooled resolved rate from 44.7% to about 49%, and gold localization raised it to 52.4%.

At the function level, the question has been tested more directly. RepairAgent [11] assumes perfect fault localization by default, and a recent survey found that 10 of 66 LLM-based repair systems make this assumption explicitly [14]. D4C [12] drops statement-level localization but still assumes the buggy function is known. It reports that letting the LLM rewrite the whole function works better than locating hunks first and fixing them afterwards. Campos et al. [13] replaced perfect function-level localization with the top-3 functions from a spectrum-based localizer, and accuracy dropped by 7.83–16.20% on a Defects4J subset.

3. The Gap

The function-level studies compare only two settings, perfect and automated localization, and they use single-function Java bugs. Repository-level repair works differently: the localizer gives the agent a set of candidate files. We still know little about how the makeup of that set affects repair. We see three open questions.

(i) Precision or recall? In the original SWE-bench experiments [3], as MAGIS [2] points out, a larger retrieval budget raised file recall from 29.58% to 51.06%, yet Claude-2's resolved ratio fell from 1.96% to 1.22%. A recent survey [14] notes the same tension: retrieval can miss the files that matter, but extra context can overflow the model's limits. Current evaluations test only a few fixed settings (none, predicted, gold) [8], so we cannot say whether a localizer should aim for a high Acc@1 or a high Recall@5.

(ii) Does change complexity matter? Multi-file fixes are harder [2], and some issues stay unsolved even when the agent receives the gold location [8]. None of the studies we reviewed checks whether good localization helps more for multi-file fixes than for single-file ones.

(iii) Are the patches actually correct? Every gain above is measured by whether a patch passes the benchmark's tests. When SWE-ABS [9] strengthened those tests, it rejected 19.71% of the previously passing patches from the top-30 SWE-bench Verified agents. So we do not know whether better localization leads to correct patches or only to patches that get past weak tests. The LLM4APR review asks for cost-aware evaluation (§9.2.2) and describes localization and validation as linked (§9.3.2) [1], but it does not answer these questions for repository-level repair.

4. Why It Matters and Proposed Study

For researchers, the answers would show which localization metric actually predicts repair success. Localizer design could then stop chasing retrieval scores that may not carry over to repair. For practitioners, localization takes about half of an agent's budget [7], so knowing how much context is "enough" shows where the cost–quality trade-off lies.

We propose a controlled study on SWE-bench Verified. The backbone LLM and the agent scaffold stay fixed. The only thing we change is the localized context the agent receives:

- We lower precision by adding distractor files to the gold file set. Distractors are not random: they are the highest-ranked non-gold files from a real localizer (CoSIL [5]) on the same issue, so they look relevant, as wrong suggestions do in practice.
- We lower recall by removing some of the gold files. Because this is only possible when a fix spans several files, the recall comparison is made within the multi-file subset only, so that its effect is not mixed up with the effect of change complexity.

Instances are grouped by change complexity, following MAGIS's regression analysis [2]. This design differs from earlier work in two ways. Loc2Repair [8] compares fixed localization conditions, and the function-level studies [11–13] work on single functions. Our study instead varies precision and recall at the repository level and checks correctness beyond test-passing. It is feasible with public benchmarks, an open-weight model (for example Qwen2.5-Coder-32B, which CoSIL [5] also uses), and a stratified subset of instances to keep the cost manageable.

5. Research Questions

RESEARCH QUESTION	INDEPENDENT VARIABLE(S)	DEPENDENT VARIABLE(S)
RQ1. How do the precision and recall of the file-level localization given to an LLM repair agent affect its resolved rate and cost on repository-level issues?	Precision {1.0, 0.5, 0.25}, set by adding distractor files taken from the top-ranked non-gold files of CoSIL [5] (all instances). Recall {1.0, ≈0.5}, set by keeping half of the gold files (rounded up, with the exact value logged per instance); compared within multi-file instances only, since a single-file instance can only have a recall of 0 or 1. Held constant: backbone LLM, scaffold, prompt, instance set.	Resolved rate (%); input/output tokens and USD cost per instance.
RQ2. Does the effect of localization quality on repair success depend on how complex the required change is?	Localization condition {none, predicted (e.g., CoSIL [5]), gold}. Moderator: complexity of the gold patch (single- vs. multi-file; number of hunks).	Resolved rate, analyzed with logistic regression that includes an IV × complexity interaction term (extending [2]).
RQ3. Does diagnostic localization (a location plus a plain-language explanation of the fault) lead to more genuinely correct patches than location-only output?	Form of the localization output {file path; file + function; diagnostic finding as in [7]}.	Resolved rate on the original tests; correctness rate under strengthened tests [9] (or manual checking of a sample); overfitting rate (patches that pass the original tests but fail the strengthened ones).

References

[1] Q. Zhang, C. Fang, Y. Xie, Y. Ma, W. Sun, Y. Yang, Z. Chen. A Systematic Literature Review on Large Language Models for Automated Program Repair. ACM TOSEM, 2026. arXiv: 2405.01466v4.

[2] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, Y. Cheng. MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue Resolution. NeurIPS, 2024. arXiv:2403.17927.

[3] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024.

[4] Z. Chen et al. LocAgent: Graph-Guided LLM Agents for Code Localization. ACL, 2025. arXiv:2503.09089.

[5] Z. Jiang, X. Ren, M. Yan, W. Jiang, Y. Li, Z. Liu. Issue Localization via LLM-Driven Iterative Code Graph Searching (CoSIL). ASE, 2025. arXiv:2503.22424.

[6] R. G. Reddy, T. Suresh, J. Doo, et al. SweRank: Software Issue Localization with Code Ranking. ICLR, 2026. arXiv:2505.07849.

[7] H. Tamoyan, S. Narenthiran, E. Arakelyan, M. Mezini, B. Ginsburg. SHERLOC: Structured Diagnostic Localization for Code Repair Agents. arXiv:2606.24820, 2026.

[8] M. N. Al Awad, S. Ivanov. Loc2Repair: A Framework for Evaluating the Impact of File-Level Issue Localization in Repo-Level LLM Repair. GeCoIn Workshop @ IJCAI-ECAI, 2026. arXiv:2606.30963.

[9] B. Yu et al. SWE-ABS: Adversarial Benchmark Strengthening Exposes Inflated Success Rates on Test-based Benchmark. arXiv:2603.00520, 2026.

[10] C. S. Xia, Y. Deng, S. Dunn, L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024.

[11] I. Bouzenia, P. Devanbu, M. Pradel. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. arXiv:2403.17134, 2024.

[12] J. Xu, Y. Fu, S. H. Tan, P. He. Aligning the Objective of LLM-based Program Repair. ICSE, 2025. arXiv:2404.08877.

[13] V. Campos, R. Shariffdeen, A. Ulges, Y. Noller. Exploring Generalizable Automated Program Repair with Large Language Models. arXiv:2506.03283, 2025.

[14] B. Yang, Z. Cai, F. Liu, B. Le, L. Zhang, T. F. Bissyandé, Y. Liu, H. Tian. A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications. arXiv:2506.23749, 2025.